Writing Database Commands to a File

(https://github.com/learn-co-curriculum/python-p3-writing-db-commands-to-a-file) (https://github.com/learn-co-curriculum/python-p3-writing-db-commands-to-a-file/issues/new)

Learning Goals

- Use SQL to store data and retrieve it later on.
- Use SQLite to build relational databases on your computer.

Key Vocab

- **SQL (Structured Query Language)**: a programming language that is used to manage relational databases and perform operations on the data that they contain.
- **Relational Database**: a collection of data that is organized in well-defined relationships. The most common type of database.
- **Query**: a statement used to return data from a database.
- **Table**: a collection of related data in a database. Composed of rows and columns. Similar to a class in Python.
- **Row**: a single record in a database table. Each column represents an attribute of the record. Similar to an object in Python.
- **Column**: a single field in a database table. Each row contains values in each column. Similar to a Python object's attributes.
- **Schema**: a blueprint of the construction of the tables in a database and how they relate to one another.

Writing SQL in a Text Editor

Up until now, we've been executing our SQL commands directly in the terminal. It is likely, however, that you will find yourself writing SQL in a file and executing that file in the context of your database. The more complex our databases become, the more tables we add and the more advanced the queries we run against them, the harder it will become to keep track of it all in the `sqlite3` prompt in our terminal.

SQL is a programming language like any other, so we can write SQL in our text editor and execute it. This allows us to keep better track of our SQL code, including the SQL statements that create tables and query data from those tables.

To write SQL in our text editor and execute that SQL against a specific database, we'll create files in our text editor that have the `.sql` extension. These files will contain valid SQL code. Then, we can execute these files *against our database* in the command line. We'll take a look at this process together in the following code along.

Code Along

Creating a Database and Table

In the terminal, create a database with the following command:

```
$ sqlite3 pets_database.db
```

Once you create your database, exit the sqlite prompt with the `.quit` command:

```
sqlite> .quit
```

Open up a text editor and create and save a file `01_create_cats_table.sql`; make sure the new file is saved in the same directory where you created the database. In this file, write your create statement:

```
CREATE TABLE cats (
  id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER
);
```

Execute that file in the command line. *Before running the command below, make sure that you've exited the SQLite prompt that you were in earlier when you created the database.*

```
$ sqlite3 pets_database.db < 01_create_cats_table.sql
```

Note: If running the above command gives you an error that the Cats table already exists, that means you created a table with that name in a previous exercise. You can enter into your Pets Database with the `sqlite3 pets_database.db` command and then remove your old table in the SQLite prompt with:

```
DROP TABLE cats;
```

Confirming Our SQL Execution

Let's confirm that the above execution of the SQL commands in our `.sql` file worked. To do so:

- In your terminal, enter into your Pets Database with the `sqlite3 pets_database.db` command.
- Then run the `.schema` command. You should see the following schema printed out, confirming that we did, in fact, create our Cats table successfully.

```
CREATE TABLE cats (
  id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER
);
```

Now, exit out of the `sqlite3` prompt with `.quit`.

Operating on our Database from the Text Editor

To carry out any subsequent actions on this database — adding a column to the cats table, dropping that table, creating a new table — we can create new `.sql` files in the text editor and execute them in the same way as above. Let's give it a shot.

To add a column to our cats table, first, create a file named `02_add_column_to_cats.sql` and fill it out with:

```
ALTER TABLE cats ADD COLUMN breed TEXT;
```

Then, execute the file in your command line:

```
$ sqlite3 pets_database.db < 02_add_column_to_cats.sql
```

Confirm that your execution of the `.sql` file worked by entering into your database in the terminal with the `sqlite3 pets_database.db` command. Once there, execute the `.schema` command and you should see that the schema of the Cats table does include the `breed` column.

▶ *When is a statement in `filename.sql` executed?*

Conclusion

You've now seen how to write your SQL code in a file, and execute the code in those files using the `sqlite3` application in the terminal. This workflow is similar to what you're used to for running Python applications by running `python filename.py` from the terminal: the program reads the content of the file (our code) and executes it. The key difference is that this time, running the code actually makes some *changes* to another file in the file system: in this case, altering the structure of the database in the `pets_database.db` file.

Resources

- **SQL Tutorial - W3Schools (https://www.w3schools.com/sql/)**
- **Documentation - SQLite (https://www.sqlite.org/docs.html)**
- **SQLite - VisualStudio Marketplace (https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite)**
- **SQLite Keywords - SQLite (https://www.sqlite.org/lang_keywords.html)**
- **ZetCode sqlite3 Tutorial (http://zetcode.com/db/sqlite/)**